



MISSION SAVOIRS

Workshop Client

MBA Développeur Full Stack

Dossier technique

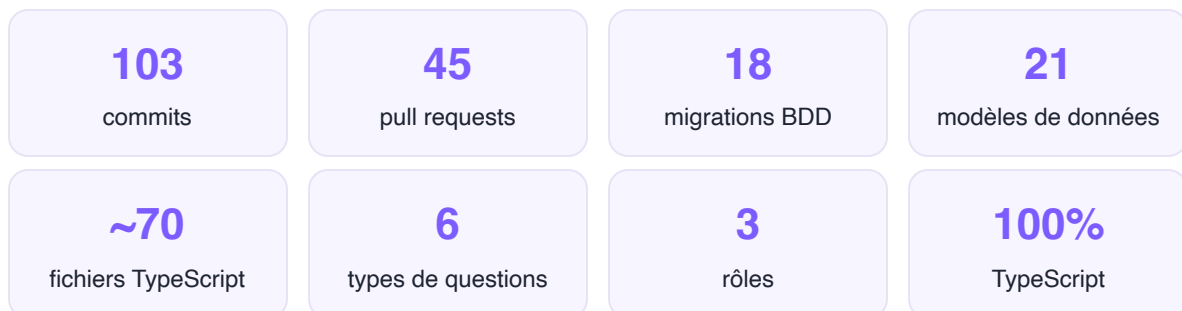
Théo Delhay

Juin 2026

Sommaire

1. [1](#). Présentation du projet — contexte, objectifs, client
2. [2](#). Cahier des charges / Note de cadrage
3. [3](#). Pilotage — comptes rendus
4. [4](#). Gestion de projet — planning, GitHub, issues, PR, Kanban
5. [5](#). Architecture technique
6. [6](#). Base de données — schéma, relations, choix
7. [7](#). Fonctionnalités
8. [8](#). Sécurité & RGPD
9. [9](#). Tests & vérifications
10. [10](#). Déploiement
11. [11](#). Difficultés rencontrées
12. [12](#). Conclusion — maintenance & évolutions

Le projet en chiffres



1. Présentation du projet

Contexte

Mission Savoirs est une **plateforme pédagogique adaptative** destinée aux élèves de l'école primaire (CP à CM2). Elle permet aux enseignants de créer des contenus (quiz, exercices interactifs) et de suivre la progression de chaque élève, tandis que les élèves avancent dans des parcours qui se débloquent progressivement et rejouent automatiquement les notions ratées.

Le projet a été mené dans le cadre du Workshop Client du MBA Développeur Full Stack, en conditions proches du réel : un client (une école), un besoin pédagogique, et un cycle de développement itératif piloté par tickets et pull requests.

Objectifs

- Proposer aux jeunes élèves une interface **simple, ludique et accessible**.
- Offrir aux enseignants un **back-office complet** : gestion des élèves, des classes/groupes, du contenu, et suivi statistique.
- Adapter le parcours : **déblocage progressif, réinjection** des questions ratées, **révisions** par niveau, **parcours personnalisés**.
- Garantir un haut niveau de **sécurité** et de **conformité RGPD**, l'application traitant des données de mineurs.

Client & acteurs

Acteur	Rôle
École Élémentaire Grand Pré Bulcos (Meylan)	Client / établissement utilisateur — responsable de traitement RGPD
Enseignant-e référent-e (Sabrina Letellier)	Porteuse du projet au sein de l'établissement, retours fonctionnels
Théo Delhay	Conception, développement full-stack, déploiement

Profils utilisateurs

Rôle	Accès
Élève (STUDENT)	Espace de jeu : thèmes, quiz, révisions, parcours personnalisés.
Professeur (TEACHER)	Back-office : son contenu, tous les élèves, classes, statistiques, validation des dessins.
Administrateur / Owner (ADMIN)	Tout le contenu, gestion des comptes staff, journal d'accès, export RGPD.

2. Cahier des charges / Note de cadrage

La note de cadrage initiale (document fourni par le client) définit le périmètre fonctionnel, les profils utilisateurs et les contraintes. Elle est jointe au présent dossier.

[Insérer ici la note de cadrage / le cahier des charges (PDF d'origine)]

Périmètre fonctionnel retenu

Domaine	Besoins
Contenu	Arborescence Thème → Sous-thème → Quiz → Questions ; 6 types de questions ; visibilité public/privé.
Élèves & classes	Comptes élèves, import en masse, classes/groupes multi-appartenance, niveaux.
Pédagogie adaptative	Déblocage progressif, réinjection des erreurs, révisions par niveau, parcours personnalisés.
Suivi	Statistiques globales et par élève, points faibles, export.
Transverse	Authentification, rôles & permissions, conformité RGPD.

3. Pilotage — comptes rendus

Le projet a été piloté de façon itérative. Chaque itération livre une fonctionnalité validée (typecheck + build + vérification end-to-end) puis intégrée via une pull request. Les comptes rendus de suivi sont consignés ci-dessous.

[Insérer ici les comptes rendus de réunion / points client (CR1, CR2, ...)]

Synthèse des itérations (extrait de l'historique réel)

Itération	Livrables
Fondations	Auth JWT, modèle de contenu, types de questions (QCM, texte, drag & drop, association, classement, dessin).
Pédagogie	Parcours personnalisés à progression isolée, révisions par niveau, tableau de bord KPI.
Multi-acteurs	Rôle professeur, propriété & cloisonnement du contenu, co-accès (partage de quiz), classes & ciblage.
Affinages	Multi-classe, pool d'élèves partagé, visibilité public/privé, import multi-groupes, thèmes partagés.
Sécurité & RGPD	Audit complet, durcissement, chiffrement des noms, journal d'accès, export, documentation de conformité.
Onboarding	Guide intégré, visite guidée, démos fidèles par catégorie, assistant local.

4. Gestion de projet

Planning & méthode

Développement itératif inspiré du flux GitHub : un ticket (issue) → une branche → une pull request → revue/validation → merge sur `main`. Chaque PR est autonome, testée et documentée. Le travail a été suivi sur un tableau Kanban (À faire / En cours / Terminé).

GitHub — dépôt

Élément	Valeur
Dépôt	<code>Distrib-ps/MISSION-SAVOIRS</code>
Branche principale	<code>main</code> (toujours déployable)
Commits	103
Pull requests fusionnées	45
Stratégie	1 fonctionnalité = 1 branche = 1 PR ; merge après typecheck + build + e2e

Kanban

[Insérer ici une capture du tableau Kanban / projet GitHub]

Issues & Pull Requests (extrait)

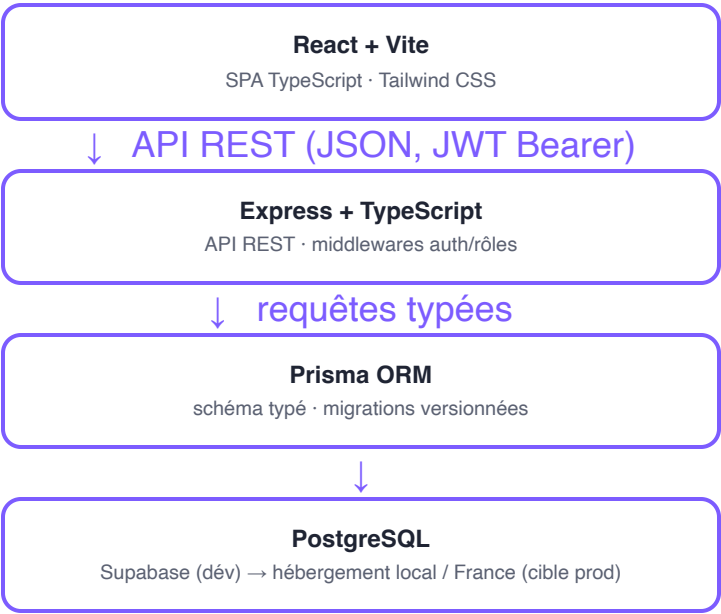
45 PR fusionnées. Extrait représentatif :

#	Pull request
16	Parcours personnalisés — progression isolée par parcours
19	Révisions par niveau — mix de questions existantes
20	KPI & tableau de bord de progression
21–25	Classes & ciblage, rôle professeur & cloisonnement, co-accès, validation des dessins
27–33	Multi-classe, pool d'élèves partagé, visibilité public/privé, import multi-groupes
34–46	Sécurité (lots 1-2) & RGPD (lots 3-5) : durcissement, chiffrement, journal, export
47–56	Onboarding : guide, visite guidée, démos fidèles par catégorie
59–63	Total élèves global, partage depuis l'arbo, questions publiques en révision, thèmes partagés, assistant d'aide

[Insérer ici une capture de la liste des Issues et/ou des PR GitHub]

5. Architecture technique

Architecture **client / serveur** découplée : une application web (SPA) communique avec une API REST, elle-même adossée à une base de données relationnelle via un ORM.



Stack & choix techniques

Couche	Technologie	Pourquoi
Front-end	React 18 + Vite, TypeScript, Tailwind CSS	DX rapide (HMR), typage de bout en bout, UI cohérente et accessible.
Back-end	Node.js + Express, TypeScript	API REST légère, middlewares clairs pour l'auth et le cloisonnement.
ORM	Prisma	Schéma unique typé, migrations versionnées, requêtes paramétrées (anti-injection).
Base de données	PostgreSQL	Relationnel robuste, contraintes d'unicité, intégrité référentielle (cascade).
Graphiques	Recharts	Visualisation des statistiques de progression.

Aspects transverses

- **Authentification** : JWT signé HS256 ; le rôle est *relu en base* à chaque requête (une rétrogradation prend effet immédiatement).
- **Permissions (RBAC)** : middlewares `authenticate`, `requireStaff` (ADMIN/TEACHER) et `requireAdmin` (Owner) ; cloisonnement métier par propriété du contenu.
- **Sécurité HTTP** : Helmet (CSP, HSTS, nosniff...), CORS restreint, limite de charge utile, anti-bruteforce sur la connexion.

- **Organisation du code** : routes par domaine (`admin/` , `student/`), moteur de quiz partagé, helpers d'ownership et de chiffrement isolés.

6. Base de données

Schéma relationnel de 21 modèles/énumérations, géré par Prisma et 18 migrations versionnées.

Entités principales

Modèle	Rôle
User	Comptes (élève / prof / admin). Noms chiffrés au repos.
Class	Classes & groupes ; relation N–N avec les élèves ; propriétaire (prof).
Theme / SubTheme	Taxonomie du contenu. Thèmes partagés ; sous-thèmes propres à chaque créateur.
Quiz / Question / Answer	Contenu pédagogique. Quiz à visibilité Public/Privé ; 6 types de questions.
QuizClass	Ciblage d'un quiz vers des classes (N–N).
QuizShare	Coaccès en lecture d'un quiz à un autre professeur.
QuizAttempt / QuestionAttempt	Tentatives et réponses des élèves (scores, indices, statut de validation des dessins).
CustomPath / CustomPathQuiz	Parcours personnalisés par élève (progression isolée).
Revision / RevisionQuestion	Révisions par niveau mixant des questions de plusieurs quiz.
AuditLog	Journal d'accès aux données sensibles (RGPD, traçabilité).

Relations & intégrité

- **Cascade** : la suppression d'un élève efface ses tentatives, réponses, dessins et parcours (conformité au droit à l'effacement).
- **SetNull** : la suppression d'un compte staff conserve le contenu créé (auteur dissocié).
- **Unicité** : nom de thème unique (taxonomie commune) ; sous-thème unique *par créateur* (`themeId`, `name`, `createdById`) ; appartenance élève↔classe en N–N.

Choix structurants

- **Thèmes partagés / sous-thèmes privés** : évite les doublons de matières (un seul « Français ») tout en laissant chaque prof organiser son propre contenu.
- **Niveau décorrélé de la classe** : un élève multi-classe a un niveau propre.
- **Chiffrement applicatif** des noms (AES-256-GCM) directement au niveau de l'ORM.

[Insérer ici le schéma relationnel (diagramme ERD Prisma / dbdiagram)]

7. Fonctionnalités

Chaque fonctionnalité est présentée par : objectif, capture (à insérer), explication.

7.1 Espace élève — jeu adaptatif

Objectif : un parcours ludique où l'élève progresse thème par thème.

[Capture : tableau de bord élève « Bonjour {prénom} ! » + thèmes]

Explication : les quiz se débloquent progressivement ($\approx 70\%$ de réussite requis). Les questions ratées sont réinjectées automatiquement plus tard dans le même sous-thème. Indice après une première erreur, solution ensuite.

7.2 Six types de questions interactives

Objectif : varier les activités selon l'apprentissage visé.

[Capture : exemples de questions (QCM, glisser-déposer, association, dessin)]

Explication : QCM, réponse libre (corrigée avec tolérance accents/casse), glisser-déposer, association, classement, et dessin (zone de dessin, validé manuellement par le prof).

7.3 Gestion du contenu (back-office)

Objectif : créer et organiser thèmes, sous-thèmes, quiz et questions.

[Capture : page Contenus avec arborescence à gauche]

Explication : navigation en arborescence, réordonnancement, visibilité Public/Privé par quiz, partage en lecture à un collègue. Les thèmes sont partagés (anti-doublon avec proposition de réutilisation).

7.4 Élèves & import en masse

Objectif : créer rapidement les comptes d'une classe.

[Capture : fenêtre d'import Excel avec aperçu valides/erreurs]

Explication : création unitaire ou import Excel/CSV (modèle téléchargeable, colonne CLASSE multi-groupes séparés par « ; »). Identifiants et mots de passe générés automatiquement et exportables. Pool d'élèves partagé entre profs (évite les doublons de comptes).

7.5 Classes & groupes (multi-appartenance)

Objectif : organiser les élèves et cibler le contenu.

[Capture : page Classes & groupes]

Explication : un élève peut appartenir à plusieurs classes/groupes ; un quiz « Privé » n'est visible que par les élèves du créateur.

7.6 Parcours personnalisés & révisions

Objectif : différencier et préparer les évaluations.

[Capture : parcours personnalisé d'un élève + création de révision]

Explication : parcours sur mesure par élève (progression isolée) ; révisions par niveau mixant des questions de plusieurs quiz, y compris des questions de quiz publics.

7.7 Validation des dessins

Objectif : corriger les réponses libres dessinées.

[Capture : page « Dessins à valider » avec  / ]

Explication : badge de notification ; l'image est purgée dès la validation (vie privée).

7.8 Statistiques de progression

Objectif : suivre élèves et contenus.

[Capture : statistiques globales + détail par élève]

Explication : indicateurs globaux, vue par élève (progression, points faibles, détail question par question), export CSV.

7.9 Onboarding intégré

Objectif : rendre un prof autonome sans formation.

[Capture : page Aide + démo « fausse page » d'une catégorie]

Explication : visite guidée au premier login, bouton « Démo » par catégorie (reproduction fidèle des écrans), guide searchable et assistant local de questions (sans IA externe).

8. Sécurité & RGPD

L'application traitant des **données de mineurs**, la sécurité et la conformité ont fait l'objet d'un audit complet (4 axes) suivi d'une remédiation en plusieurs lots, puis d'un *second audit de vérification*.

Authentification & mots de passe

- Mots de passe **hachés bcrypt** (coût 12) ; jamais renvoyés par l'API.
- JWT **HS256**, secret obligatoire (refus de démarrage si secret par défaut), algorithme épinglé.
- Rôle **relu en base** à chaque requête (révocation immédiate).
- **Anti-bruteforce** sur la connexion (limite par IP + identifiant) avec message de déblocage par l'enseignant.

Permissions & cloisonnement

- RBAC à 3 niveaux ; un professeur ne gère que **son** contenu (propriété `createdById`).
- Données élèves scopées sur l'utilisateur du token (pas de fuite inter-élèves).
- **IDOR fermés** : réponses aux questions, révisions et parcours validés contre la propriété / l'accessibilité.

Protection des entrées

- Requêtes Prisma **paramétrées** (aucune SQL brute) ; pas de rendu HTML non échappé côté client.
- Upload borné (taille + type), validation stricte des dessins (data-URL image + plafond).
- En-têtes Helmet, CORS restreint ; dépendances assainies (remplacement de la lib `xlsx` vulnérable).

RGPD

Exigence	Mise en œuvre
Minimisation	Aucun email/date de naissance/géolocalisation d'élève. Données strictement pédagogiques.
Chiffrement au repos	Noms (prénom/nom) chiffrés AES-256-GCM via extension Prisma (clé en variable d'environnement).
Conservation	Dessins purgés dès validation ; résultats supprimés au départ de l'élève (cascade).
Droits des personnes	Export des données d'un élève (accès/portabilité) réservé à l'administrateur.
Traçabilité	Journal d'accès (consultations, exports, suppressions).
Transparence	Pages Confidentialité & Mentions légales ; documentation (registre art. 30, note parents, AIPD, DPA).
Hébergement	Cible : local, en France — aucun transfert hors UE, aucun sous-traitant tiers.

9. Tests & vérifications

La stratégie de qualité repose sur trois piliers, appliqués à **chaque pull request** :

1. **Typage statique strict** — TypeScript de bout en bout, `tsc --noEmit` sans erreur (front + back).
2. **Build de production** — `vite build` vérifié systématiquement.
3. **Vérifications fonctionnelles end-to-end** — scripts exécutés contre le serveur réel (JWT forgé, appels API, contrôle en base, puis nettoyage des données de test).

Campagnes de vérification end-to-end exécutées

Scénario	Vérifie	Résultat
Multi-classe	Élève dans 2 classes ; login/«me» renvoient les classes ; retrait d'une classe	✓
Import multi-groupes	Import CSV & XLSX rattache l'élève à plusieurs groupes ; type invalide rejeté	✓
Pool d'élèves partagé	Un prof voit/édite tout élève ; ne peut pas toucher un compte staff (403)	✓
Visibilité quiz	Privé visible par les élèves du créateur uniquement ; <code>/start</code> 200/403	✓
Durcissement (lot 1)	En-têtes Helmet ; rate-limit 8→429 ; IDOR réponse élève 403	✓
IDOR contenu	Révision / parcours avec contenu d'autrui → 403	✓
Journal & export	Export 200 + action tracée ; journal réservé à l'Owner (prof 403)	✓
Chiffrement des noms	Base = chiffré, API = clair ; login & recherche OK	✓
Export RGPD owner-only	Admin 200, prof 403	✓
Total élèves global	Le tableau de bord prof affiche le total global	✓
Questions publiques en révision	Question publique 201, question privée d'autrui 403	✓
Thèmes partagés	Thème visible par tous ; sous-thème homonyme par autre prof 201 ; suppression d'autrui 403	✓

12 campagnes (≈ 35 assertions), toutes validées. Typecheck et build verts sur l'ensemble des 45 PR fusionnées.

Axe d'amélioration assumé : ces vérifications sont scriptées manuellement. La mise en place d'une suite automatisée (Vitest/Jest + Supertest) et d'une CI (GitHub Actions) est identifiée comme évolution prioritaire (cf. section 12).

10. Déploiement

Topologie

- **Front** : application statique (build Vite) servie par l'hébergeur.
- **API** : service Node/Express derrière un reverse-proxy HTTPS (`trust proxy` activé).
- **Base** : PostgreSQL — Supabase en développement ; cible production : hébergement local / en France.

Variables d'environnement

Variable	Rôle
<code>DATABASE_URL</code>	Connexion runtime à PostgreSQL.
<code>DIRECT_URL</code>	Connexion directe pour les migrations Prisma.
<code>JWT_SECRET</code>	Secret de signature des jetons (fort, propre à l'environnement — démarrage refusé sinon).
<code>ENCRYPTION_KEY</code>	Clé AES-256 (32 octets base64) du chiffrement des noms. À sauvegarder précieusement.
<code>CLIENT_URL</code>	Origine autorisée (CORS).
<code>PORT</code>	Port d'écoute de l'API.

Procédure

1. Configurer les variables d'environnement (jamais commitées — `.env` gitignoré).
2. Appliquer les migrations : `prisma migrate deploy` (sans reset).
3. Builder le front (`vite build`) et démarrer l'API.
4. Première mise en production : exécuter le script de chiffrement des noms existants.

11. Difficultés rencontrées

Migrations sur base partagée sans reset

Problème : la base de développement est partagée ; `prisma migrate dev` proposait un reset destructeur (drift). **Solution** : génération du SQL via `migrate diff` puis application avec `migrate deploy` (jamais de reset), et usage du binaire local de Prisma (la version globale étant incompatible).

Chiffrement des noms vs recherche/tri

Problème : une fois les noms chiffrés, la recherche et le tri SQL par nom ne fonctionnent plus (le texte est illisible en base). **Solution** : extension Prisma chiffrant à l'écriture / déchiffrant à la lecture (y compris les relations imbriquées), et déplacement de la recherche/du tri côté applicatif après déchiffrement.

Dépendance vulnérable (xlsx)

Problème : la librairie `xlsx` présente des failles sans correctif, sur des fichiers uploadés. **Solution** : remplacement par `exceljs` (front + back), avec chargement dynamique côté client pour ne pas alourdir le bundle.

Modèle de contenu : doublons de thèmes

Problème : nom de thème unique en base + cloisonnement par prof = le 2^e prof ne pouvait ni voir ni recréer « Français ». **Solution** : thèmes *partagés* (visibles par tous) avec proposition de réutilisation, et sous-thèmes uniques par créateur (migration de contrainte).

Faillles débusquées lors de l'audit de vérification

Le second audit a révélé des **IDOR** dans la composition des révisions et parcours (un prof pouvait inclure le contenu d'un autre). Corrigés par un contrôle de propriété / d'accessibilité, puis re-vérifiés (403 attendu).

Demos : fidélité sans appel réseau

Problème : guider l'utilisateur sur les écrans réels impliquait d'attendre l'apparition d'éléments conditionnels.

Solution : démos jouées sur une « fausse page » reproduisant fidèlement le markup réel avec des données en dur — fluide, sans données créées.

12. Conclusion

Mission Savoirs est une application full-stack **complète et cohérente** : pédagogie adaptative côté élève, back-office riche côté enseignant, et une attention forte portée à la sécurité et à la conformité RGPD — exigence renforcée par le traitement de données de mineurs. Le projet a été conduit avec une discipline d'intégration continue manuelle (typecheck, build, vérifications e2e à chaque PR).

Maintenance

- Code 100 % TypeScript, schéma de données unique et migrations versionnées : évolutions traçables et sûres.
- Documentation : guide enseignant, dossier RGPD (registre, AIPD, DPA), manuel d'aide intégré.
- Variables sensibles externalisées ; sauvegarde de la clé de chiffrement documentée.

Évolutions envisagées

- **Tests automatisés & CI** : suite Vitest/Jest + Supertest, GitHub Actions (priorité).
- **Sessions** : passage du JWT en cookie `HttpOnly` + jeton de rafraîchissement.
- **Contenu** : sous-thèmes partagés, catalogue public de quiz, duplication entre profs.
- **Assistant** : éventuel chatbot IA hébergé en UE (sous réserve de mise à jour RGPD).
- **Finalisation conformité** : signature du DPA, AIPD validée, hébergement local en production.